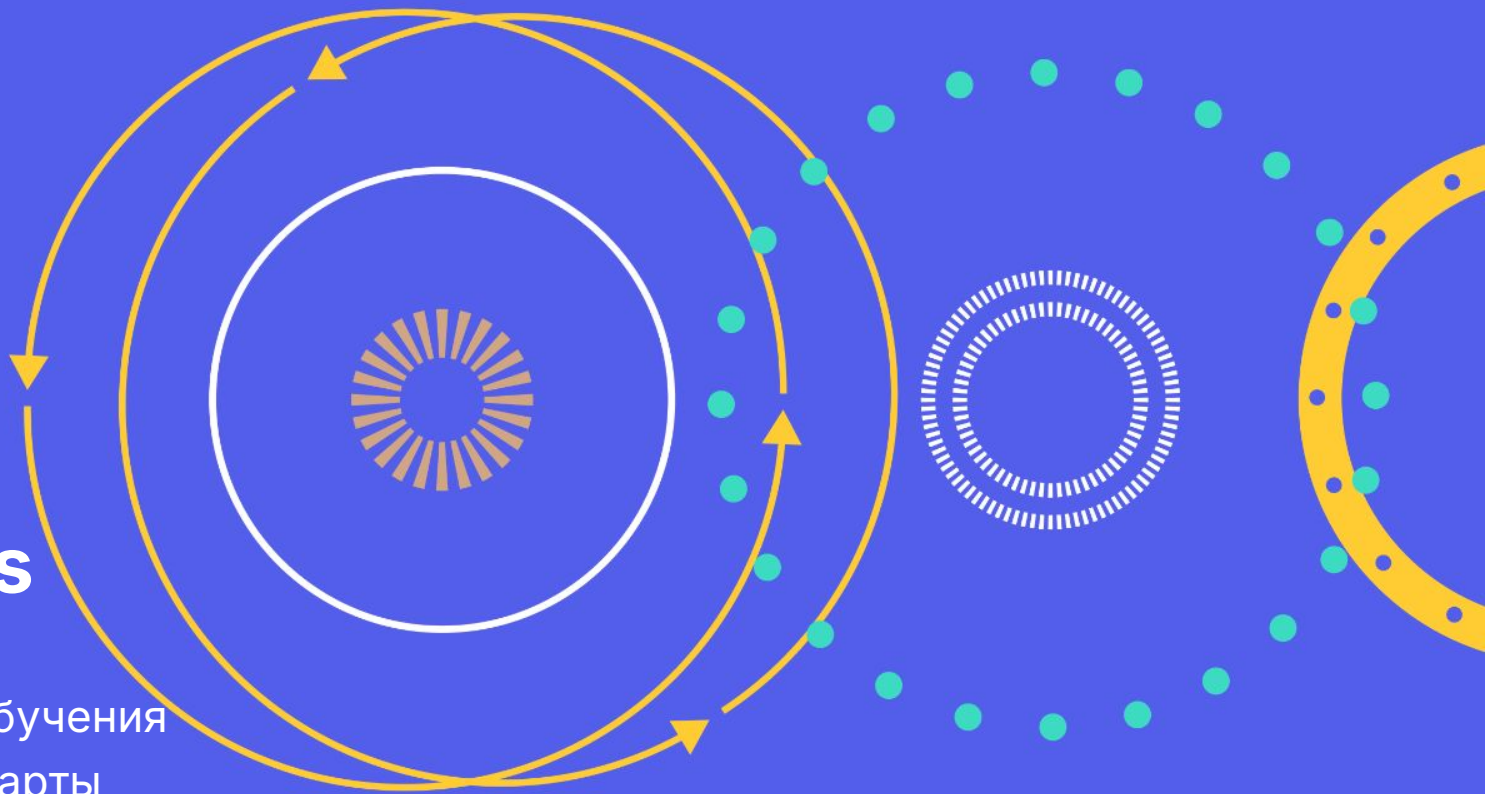


AIRI SUMMER 2026

LLM-Guided Performance Engineering for GPU Kernels

Может ли небольшая языковая модель без дообучения сама написать быструю программу для видеокарты

Данил Сивцов · Булат Шарипов · Тимур Пономарев



Эффективность GPU-ядер определяет скорость нейросети

Что такое ядро

Небольшая программа, которая выполняет одну операцию нейросети прямо на видеокарте — например, перемножает матрицы.

Почему эффективные ядра важны

В зависимости от степени оптимизации работы с памятью, скорости ядер могут отличаться в разы для одной математической операции

Проблема

Быстрое ядро пишет инженер, знающий и модель, и конкретную видеокарту. Это дорогой и медленный труд.

Поэтому «объединенные» ядра, где целый слой трансформера считается одним блоком, редко пишут вручную — из-за сложности. **Именно там и остаётся неиспользованный запас скорости.**

Постановка



1 · Модель-агент

Использовали DeepSeek v4 Flash, Qwen3.6-35B-A3B и Gemma4-31B-it.

2 · Задачи и инфраструктура

Взяли формат KernelBench, но задач под CuTe и FP8 в нём нет — написали свои бейзлайны, входы и пороги расхождения. Оценка на сервере NVIDIA B300.

3 · Фреймворк агента

Старт — opencode в headless-режиме, затем перенесли всё в KernelEvo и интегрировали оценщик B300.

Документация и инструменты для агента

E1: какая документация позволяет агенту решать задачи

Задача	Только задача	+ описание библиотеки	+ примеры кода	+ разбор ошибок
Масштабирование FP4	0/6	4/6	6/6	6/6
Нормализация слоя	0/6	6/6	5/6	5/6
Матрицы + LeakyReLU	0/6	3/6	5/6	5/6
Матрицы + деление	0/6	0/6	6/6	6/6
Матрицы + ReLU	0/6	0/6	5/6	6/6
Итого решено	0/30	13/30	27/30	28/30
Токенов на решение	—	3,88 млн	1,79 млн	1,19 млн

E2–E4: три новых инструмента

Задача	Опора	Справка — в запрос	Агент-рецензент	Сводка профиля
Масштабирование FP4	6/6	6/6	4/6	5/6
Нормализация слоя	5/6	5/6	5/6	6/6
Матрицы + LeakyReLU	5/6	6/6	6/6	6/6
Матрицы + деление	6/6	6/6	5/6	6/6
Матрицы + ReLU	6/6	4/6	4/6	5/6
Итого решено	28/30	27/30	24/30	28/30
Токенов на решение	1,19 млн	1,67 млн	1,74 + 0,38	1,21 млн

Вывод для обвязки: **примеры кода и разбор ошибок, справка файлами по требованию, без рецензента** . Именно эта конфигурация переносится на настоящий слой.

Оптимизация целого трансформерного блока

Блок	Сводка профилировщика	Успешных ходов	Лучшее время	Ускорение	Вход, токенов
GPT-2 Small	выключена	4/6	0,5205 мс	6,90×	6,83 млн
эталон 3,594 мс	стандартная	8/9	0,8295 мс	4,33×	10,91 млн
	с таймлайном	8/9	0,3164 мс	11,35×	12,42 млн
Qwen3.5-4B	выключена	6/9	0,6262 мс	79,67×	14,66 млн
эталон 49,885 мс	стандартная	8/9	51,1809 мс	0,97×	10,65 млн
	с таймлайном	8/9	7,3564 мс	6,78×	12,98 млн

Целый блок GPT-2 доведён с 3,594 до **0,3164 мс (11,35×**), блок Qwen3.5-4B — с 49,885 до **0,6262 мс (79,67×**) .
Весь код полностью сгенерирован агентами

Сжатый контекст: корректность и стоимость

Compact vs Full · Qwen · 8 цепочек на задачу · до 6 ходов		
Задача или метрика	Compact	Full
Масштабирование FP4	7/8	8/8
LayerNorm FP8	8/8	8/8
ConvTranspose3D FP8	2/8	0/8
Решено цепочек	17/24	16/24
Success@1, macro	45,8%	54,2%
Success@6, macro	70,8%	66,7%
Входных токенов	273 052	2 191 414

Сжатие контекста не влияет на корректность

Контроль релевантности · одинаковый размер контекста (±0,1%)			
Задача	Task-only	Matched-irrelevant	Correct compact
Масштабирование FP4	0/8	0/8	7/8
LayerNorm FP8	0/8	0/8	7/8
GEMM + LeakyReLU FP8	0/8	0/8	0/8
Итого	0/24	0/24	14/24

Важен именно релевантный контекст, просто документация не помогает

Динамический RAG и out-of-sample GPT-2

Error-conditioned RAG · 6 задач · 12 runs и 36 attempts на arm						
Модель	Arm	Runs	Attempts	Вход, ток.	Tools	B300
Qwen	Compact	6/12	18/36	13,40 млн	999	64
Qwen	Compact + RAG	9/12	20/36	17,76 млн	1185	80
Gemma	Compact	9/12	18/36	8,51 млн	546	85
Gemma	Compact + RAG	2/12	5/36	8,16 млн	628	125

RAG не дал воспроизводимого выигрыша

Out-of-sample GPT-2 Small · 3 persistent-сессии · до 20 evaluations					
Модель	Сессий	PASS	Eval	Вход, ток.	Выход, ток.
Gemma	3	0/3	50	6 891 471	169 566
Qwen	3	0/3	5	17 449 878	594 051

Gemma почти справилась с задачей, Qwen не приблизился к решению



Результаты:

- Показано влияние различных харнесов на генерацию
- Разработан и предложен программный харнес для CuTe и FP8, получили ускорение в среднем в 8 раз
- Показано как сжатие контекста приводит к уменьшению потребления токенов

Спасибо

Данил Сивцов · Булат Шарипов · Тимур Пономарев



@airi_research_institute



airi.net